

docker

parte 3: Docker Compose

Lo que ya sabemos

- Semana 1: fundamentos, `docker run`, `ps`, `logs`, `exec -it` ...
- Semana 2: `Dockerfile`, `docker build`, tags, volúmenes, redes básicas, Docker Hub.
- Hoy: ¿y si mi aplicación necesita **varios contenedores a la vez** (app + base de datos, por ejemplo)?

El problema de las apps reales

Una aplicación web típica no es un único contenedor:

- Un contenedor para la **app** (backend).
- Un contenedor para la **base de datos**.
- Quizá uno más para una **caché** (Redis)...

Levantarlos todos a mano con `docker run`, configurando red y orden, es tedioso y propenso a errores.

Un ejemplo real: ¿cuántas imágenes necesitas?

Imagina que trabajas en **3 proyectos** que necesitan estos 5 servicios en ejecución:

- mariadb , redis , mongo , nginx , php-fpm

Además, cada uno tiene una imagen distinta para **desarrollo** y para **producción**.

Y eso se multiplica rápido

En este escenario tan habitual podrías acabar con:

$3 \text{ proyectos} \times 5 \text{ servicios} \times 2 \text{ entornos (dev/prod)} = 30 \text{ imágenes Docker}$

...cada una con sus propios parámetros de arranque, variables y versiones.

Arrancar cada contenedor a mano con `docker run` para configurar cada proyecto se vuelve, sencillamente, inviable.

¿Qué es Docker Compose?

Docker Compose permite definir, en un único fichero, todos los servicios (contenedores) de una aplicación y levantarlos todos juntos con un solo comando.

```
docker compose up -d
```

`docker-compose.yml`

- Fichero en formato **YAML**.
- Se coloca en la raíz del proyecto.
- Define `services`, y opcionalmente `volumes` y `networks` de nivel superior.

YAML en 30 segundos

- La jerarquía se marca con **indentación** (espacios, nunca tabuladores).
- Las listas se marcan con `-`.
- Es sensible a cómo se indenta: un espacio de más o de menos puede romperlo.

Estructura mínima

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secreto
```

Dos servicios: `web` y `db` .

image vs build

```
services:
  db:
    image: postgres:16          # imagen ya publicada

  app:
    build: ./app                # construir desde un Dockerfile propio
```

- **image** : usa una imagen existente (Docker Hub u otro registry).
- **build** : construye la imagen a partir de un **Dockerfile** en esa carpeta.

ports — publicar puertos

```
services:  
  web:  
    image: nginx  
    ports:  
      - "8080:80"
```

Igual que `-p 8080:80` en `docker run` : puerto del **host** : puerto del **contenedor**.

environment — variables de entorno

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: secreto
      POSTGRES_DB: mi_app
```

También se puede usar como lista: `– POSTGRES_PASSWORD=secreto` .

Ficheros `.env`

```
# .env  
DB_PASSWORD=secreto
```

```
environment:  
  POSTGRES_PASSWORD: ${DB_PASSWORD}
```

Compose sustituye `${DB_PASSWORD}` por el valor del `.env`. Así evitamos escribir contraseñas directamente en el YAML.

volumes — persistir datos

```
services:
  db:
    image: postgres:16
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

`db_data` es un **volumen con nombre**, gestionado por Docker: los datos sobreviven a `docker compose down`.

Bind mount para desarrollo

```
services:  
  app:  
    build: ./app  
    volumes:  
      - ./app:/code
```

Monta el código del host dentro del contenedor: los cambios se ven al instante, sin reconstruir la imagen.

Redes: lo que Compose hace gratis

- Al ejecutar `docker compose up`, Compose crea **automáticamente una red** para el proyecto.
- Todos los servicios se conectan a ella.
- **No hace falta** publicar puertos para que los servicios se vean entre sí.

Comunicación entre servicios: por nombre

```
services:  
  web:  
    build: ./app  
  db:  
    image: postgres:16
```

Desde `web`, la base de datos se referencia como `db` (su nombre de servicio), **no** como `localhost`:

```
DATABASE_URL=postgres://user:pass@db:5432/mi_app
```

`depends_on` — orden de arranque

```
services:
  app:
    build: ./app
    depends_on:
      - db
  db:
    image: postgres:16
```

Arranca primero `db`, después `app`. **Ojo:** no garantiza que `db` esté ya lista para aceptar conexiones, solo que el contenedor ha arrancado.

El problema del "arrancado pero no listo"

- Un contenedor de base de datos puede tardar unos segundos en aceptar conexiones tras arrancar.
- Soluciones reales: reintentos en la propia app, o un `healthcheck` que compruebe de verdad el estado del servicio.

docker compose up / down

```
docker compose up -d          # crea y arranca todo, en segundo plano
docker compose up --build     # reconstruye imágenes con `build` antes de arrancar
docker compose down           # para y elimina contenedores + red del proyecto
docker compose down -v        # además elimina los volúmenes con nombre
```

stop / start / restart

```
docker compose stop      # para los contenedores, sin eliminarlos
docker compose start     # los vuelve a arrancar
docker compose restart   # para y arranca de nuevo
```

down elimina; **stop / start** conservan todo para retomarlo después.

Inspeccionar el stack

```
docker compose ps          # estado de los servicios del proyecto
docker compose logs        # logs de todos los servicios
docker compose logs -f db  # logs en tiempo real de un servicio concreto
```

Entrar en un servicio

```
docker compose exec app bash      # sesión en un contenedor YA en marcha
docker compose run app python migrate.py  # contenedor NUEVO y puntual
```

- `exec` : para un servicio que ya está corriendo.
- `run` : para lanzar algo puntual (p. ej. una migración) sin tocar el contenedor en producción.

Ejemplo completo: app + base de datos

```
services:
  app:
    build: ./app
    ports:
      - "3000:3000"
    environment:
      DATABASE_URL: postgres://user:pass@db:5432/mi_app
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mi_app
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Buenas prácticas

- Fija versiones concretas (`postgres:16` , no `latest`).
- No publiques puertos que no necesiten estar accesibles desde fuera (p. ej. la base de datos).
- Usa `.env` para credenciales, nunca las escribas directamente en el YAML.
- Usa volúmenes con nombre para cualquier dato que deba sobrevivir a `down` .

Ejemplos reales para inspirarte

No hace falta escribir siempre un `docker-compose.yml` desde cero: Docker mantiene un repositorio con ejemplos oficiales y reales, `docker/awesome-compose` :

- **WordPress:** `github.com/docker/awesome-compose/blob/master/official-documentation-samples/wordpress/README.md`
- **Django:** `github.com/docker/awesome-compose/blob/master/official-documentation-samples/django/README.md`
- **nginx + Node.js + Redis:** `github.com/docker/awesome-compose/tree/master/nginx-nodejs-redis`

¿Dónde consultar todas las claves disponibles?

Lo visto hoy (`services`, `image`, `build`, `ports`, `environment`, `volumes`, `networks`, `depends_on` ...) es solo una parte de lo que admite Compose.

Referencia oficial del fichero Compose:

`docs.docker.com/reference/compose-file/`

Ahí están documentadas también claves más avanzadas (`healthcheck`, `profiles`, `deploy` ...) que no cubrimos hoy.

Y esto, ¿para qué sirve en despliegue real?

- Hoy lo usamos en local, pero el mismo `docker-compose.yml` (con ajustes) puede levantar un stack en un servidor real.
- Es la base conceptual de herramientas de **orquestación** más avanzadas (Kubernetes, Docker Swarm) cuando se necesita escalar a muchas máquinas.
- Cierra el círculo del módulo: de `git` (versionar código) a `docker` / `compose` (empaquetar y desplegar la aplicación completa).

Comandos que veremos hoy

- `docker compose up -d`, `docker compose down`
- `docker compose build`, `docker compose ps`
- `docker compose logs -f`, `docker compose exec`, `docker compose run`

A practicar

Tu primer stack multi-contenedor